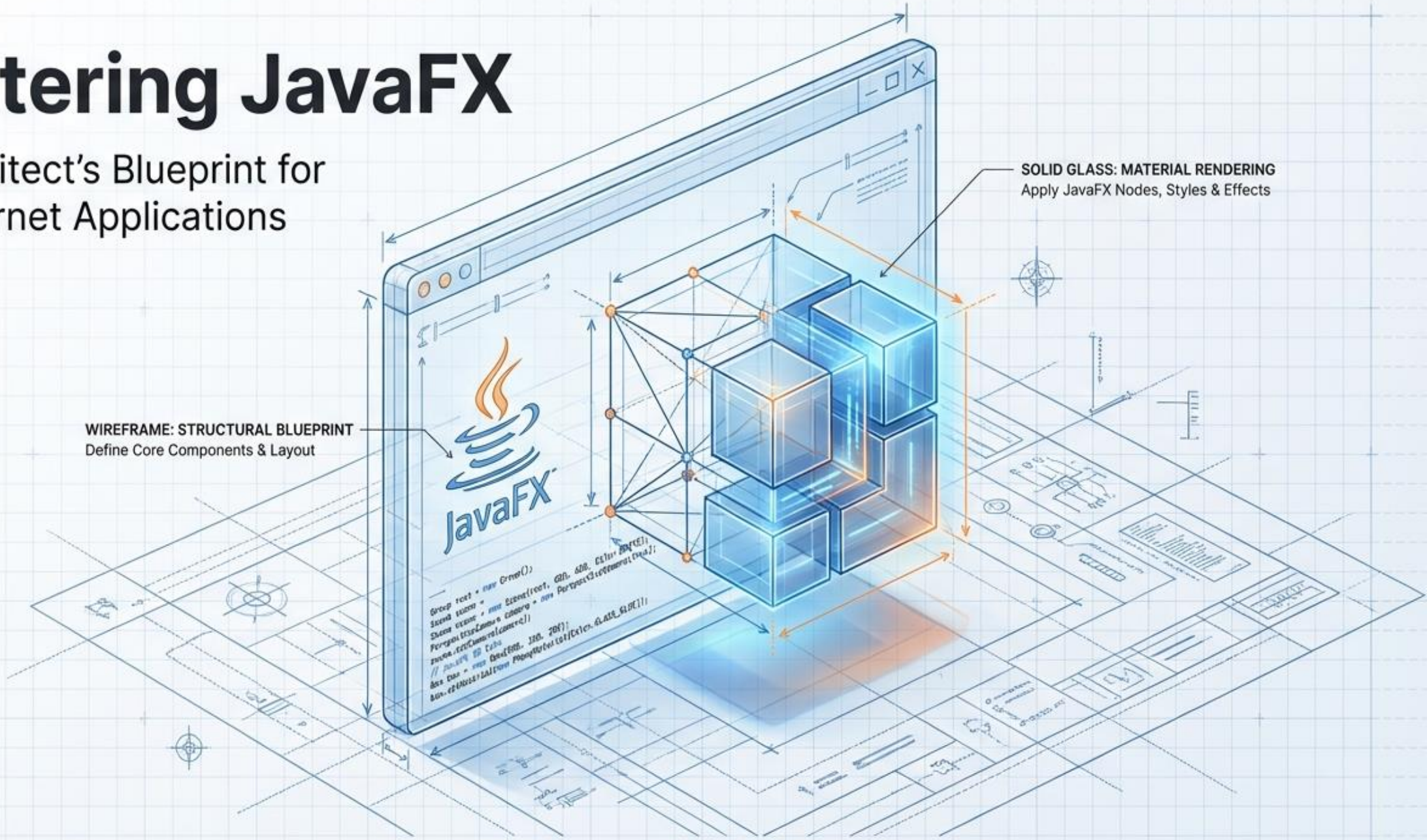
Mastering JavaFX

The Architect's Blueprint for Rich Internet Applications



Synthesized insights from GeeksforGeeks, TutorialsPoint, Bro Code, and xemacscode

Advanced Programming Applications

Lecture 04- JavaFX – Part I

Dr. Mohamed Mostafa Eltaweel

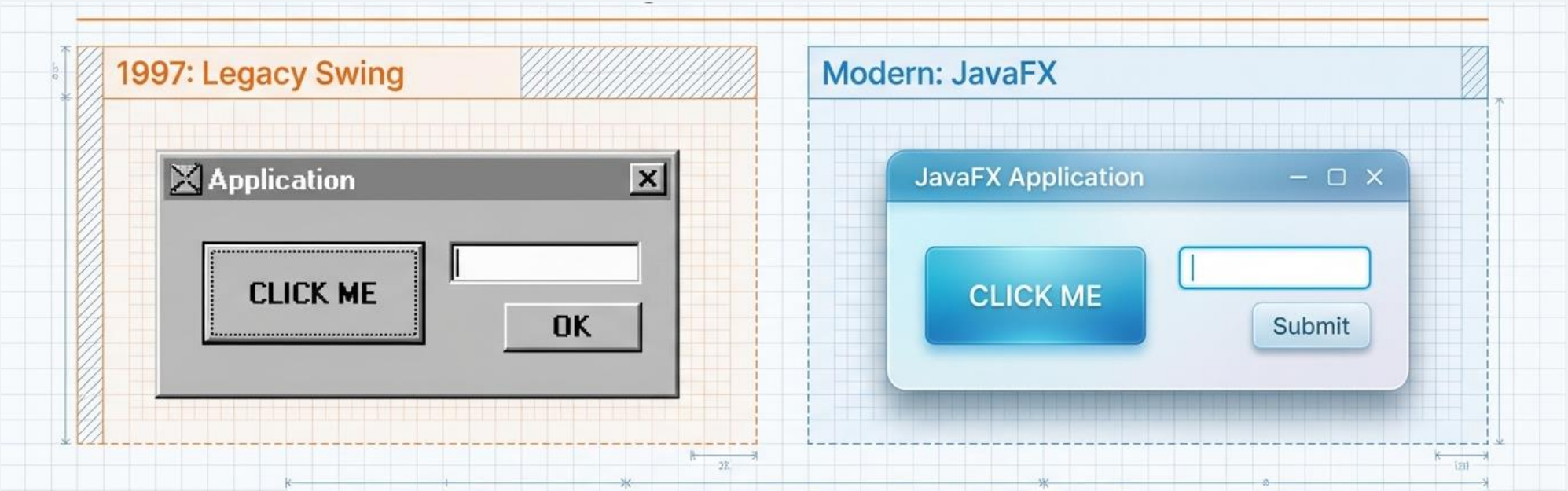
Lecture Outline

1. What are JavaFX?
2. JavaFX vs Legacy Swing
3. JavaFX Architecture
4. JavaFX Basic Application Structure
5. Preparing a Scene
6. Important packages of JavaFX
7. Panes, UI Controls, and Shapes
8. Practice

What are JavaFX?

- **JavaFX** is a Java library (complete API) used to build Rich Internet Applications to build GUI applications with rich graphics..
- The applications written using this library can run consistently across multiple platforms.
- The applications developed using JavaFX can run on various devices such as Desktop Computers, Mobile Phones, TVs, Tablets, etc.
- It replaced the use of **Advanced Windowing Tool Kit** and **Swing** Libraries.

JavaFX vs Legacy Swing



Architecture	Legacy Swing	Modern JavaFX
Architecture	Partial MVC	Consistent MVC Pattern
Styling	Pluggable Look & Feel	Pluggable Look & Feel
Performance	Software Rendered	Software Rendered
Input	Mouse/Keyboard	Mouse/Keyboard

	Legacy Swing	Modern JavaFX
Architecture	Partial MVC	Consistent MVC Pattern
Styling	Pluggable Look & Feel	CSS & FXML Declarative
Performance	Software Rendered	Hardware Accelerated (GPU)
Input	Mouse/Keyboard	Multi-touch & Gestures

JavaFX Architecture

JavaFX follows a **Scene Graph architecture**.

- **Core Components:**
  - **Stage**
  - **Scene**
  - **Nodes**

```
Stage (Window)
   → Scene
        → Layout Pane
             → Controls (Button, Label, etc.)
```

JavaFX Basic Application Structure

```
import javafx.application.Application;
import javafx.stage.Stage;

public class Main extends Application {

    @Override
    public void start(Stage primaryStage) {
        primaryStage.setTitle("My First JavaFX App");
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

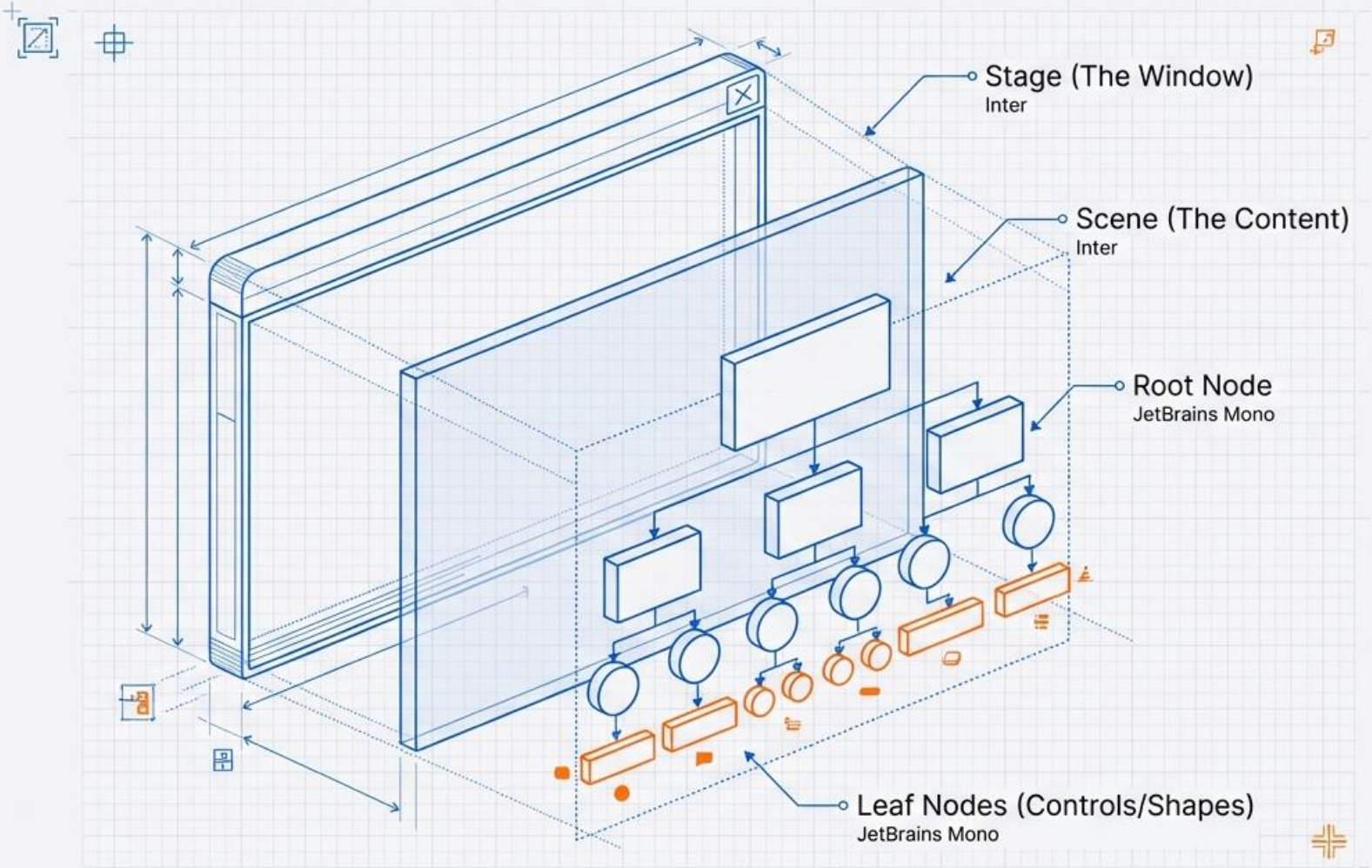
base class

window

entry point

starts JavaFX runtime

JavaFX Hierarchy



Stage, Scene, and Nodes

1. Stage

- Represents the window. A stage has two parameters determining its position namely **Width** and **Height**.
- You have to call the show() method to display the contents of a stage.

```
primaryStage.setWidth(400);
primaryStage.setHeight(300);
```

2. Scene

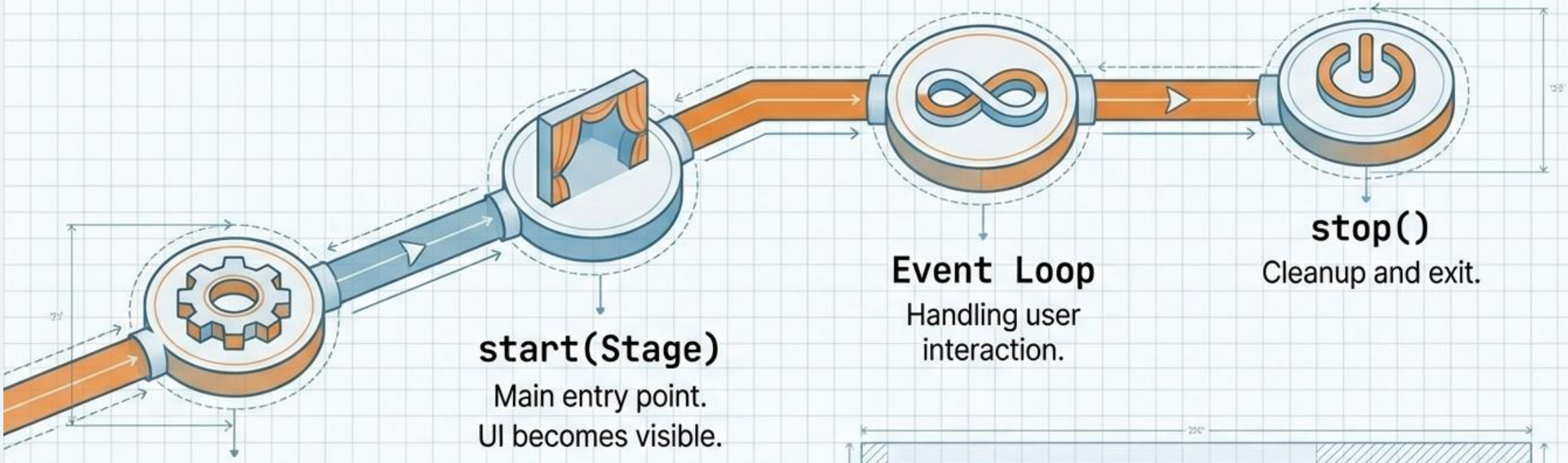
- A tree-like data structure (hierarchical) representing Container for all content.

```
Scene scene = new Scene(root, 400, 300);
primaryStage.setScene(scene);
```

- **3. Nodes**
  - A visual/graphical object of a scene graph. Everything in JavaFX is a **Node**:

    Button, Label, TextField, Layout panes, Shapes

The Application Lifecycle



`init()`
Load resources.
Non-GUI thread.

`start(Stage)`
Main entry point.
UI becomes visible.

`Event Loop`
Handling user interaction.

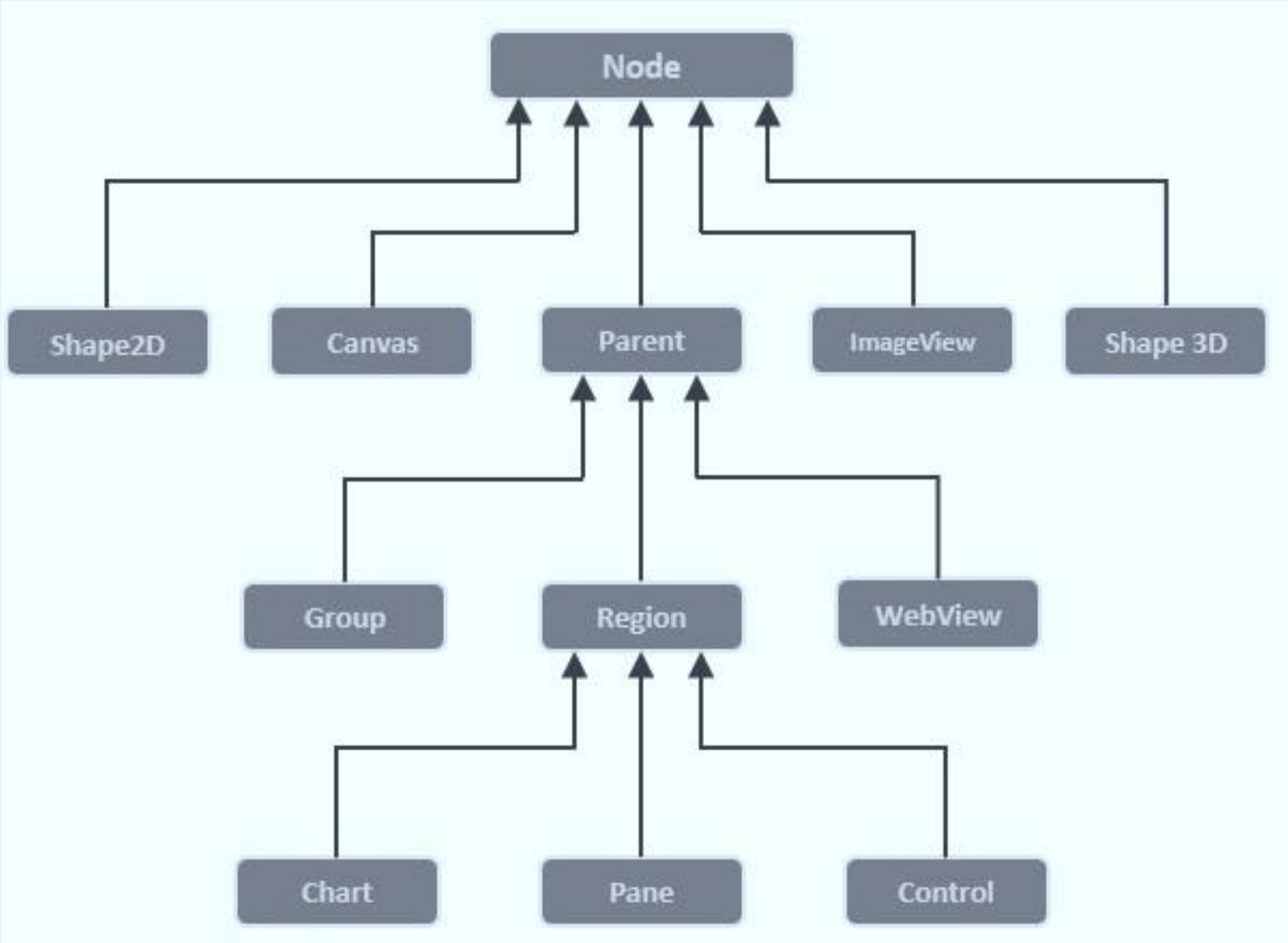
`stop()`
Cleanup and exit.

The Theater Metaphor

The "Stage" is the window, the "Scene" is the content container, and the "Nodes" are the actors.

Preparing a Scene

To build a JavaFX application, you must create a **scene graph** with the required nodes. The first node is the **root node**, which serves as the top-level container. The root can be a **Group, Region, or WebView**.

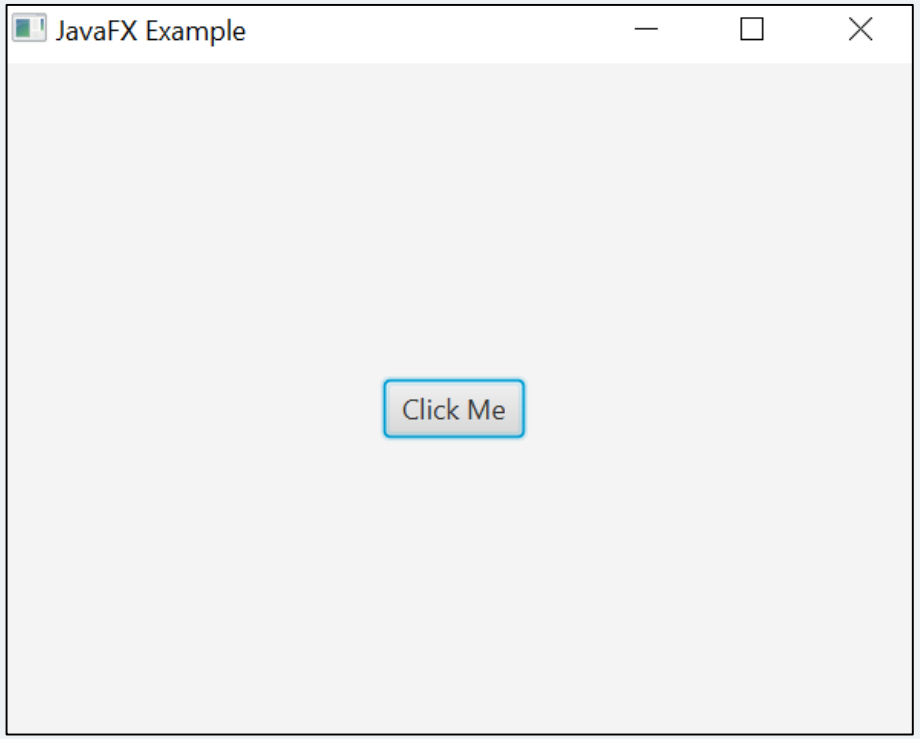


Preparing a Scene

- **Group** − A Group node is represented by the class named **Group** which belongs to the package **javafx.scene**, you can create a Group node by instantiating this class `Group root = new Group()`

```
@Override
public void start(Stage stage) {

TextField t1 = new TextField();
t1.setLayoutX(100); t1.setLayoutY(50);

TextField t2 = new TextField();
t2.setLayoutX(100); t2.setLayoutY(90);

Button btn = new Button("Submit");
btn.setLayoutX(120); btn.setLayoutY(130);

Group root = new Group(t1, t2, btn);
Scene scene = new Scene(root, 400, 250);

stage.setScene(scene);
stage.setTitle("Form");
stage.show();
}

public static void main(String[] args) {
launch();
}
}
```

```java
import javafx.application.Application; import javafx.scene.Scene;
import javafx.scene.control.Button; import javafx.scene.layout.StackPane;
import javafx.stage.Stage; import javafx.scene.paint.Color;

public class HelloFX extends Application {
    @Override
    public void start(Stage stage) {

        Button btn = new Button("Click Me");
        StackPane root = new StackPane();
        root.getChildren().add(btn);

        Scene scene = new Scene(root, 400, 300);

       //setting color to the scene
       scene.setFill(Color.BROWN);

        stage.setTitle("JavaFX Example");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```



Important packages of JavaFX

- **javafx.application** − Contains a set of classes responsible for the JavaFX application life cycle.
- **javafx.animation** − Contains classes to add transition based animations such as fill, fade, rotate, scale and translation, to the JavaFX nodes.
- **javafx.css** − Contains classes to add CSSlike styling to JavaFX GUI applications.
- **javafx.event** − Contains classes and interfaces to deliver and handle JavaFX events.
- **javafx.geometry** − Contains classes to define 2D objects and perform operations on them.
- **javafx.stage** − This package holds the top level container classes for JavaFX application.
- **javafx.scene** − This package provides classes and interfaces to support the scene graph. In addition, it also provides sub-packages such as canvas, chart, control, effect, image, input, layout, media, paint, shape, text, transform, web, etc. There are several components that support this rich API of JavaFX.

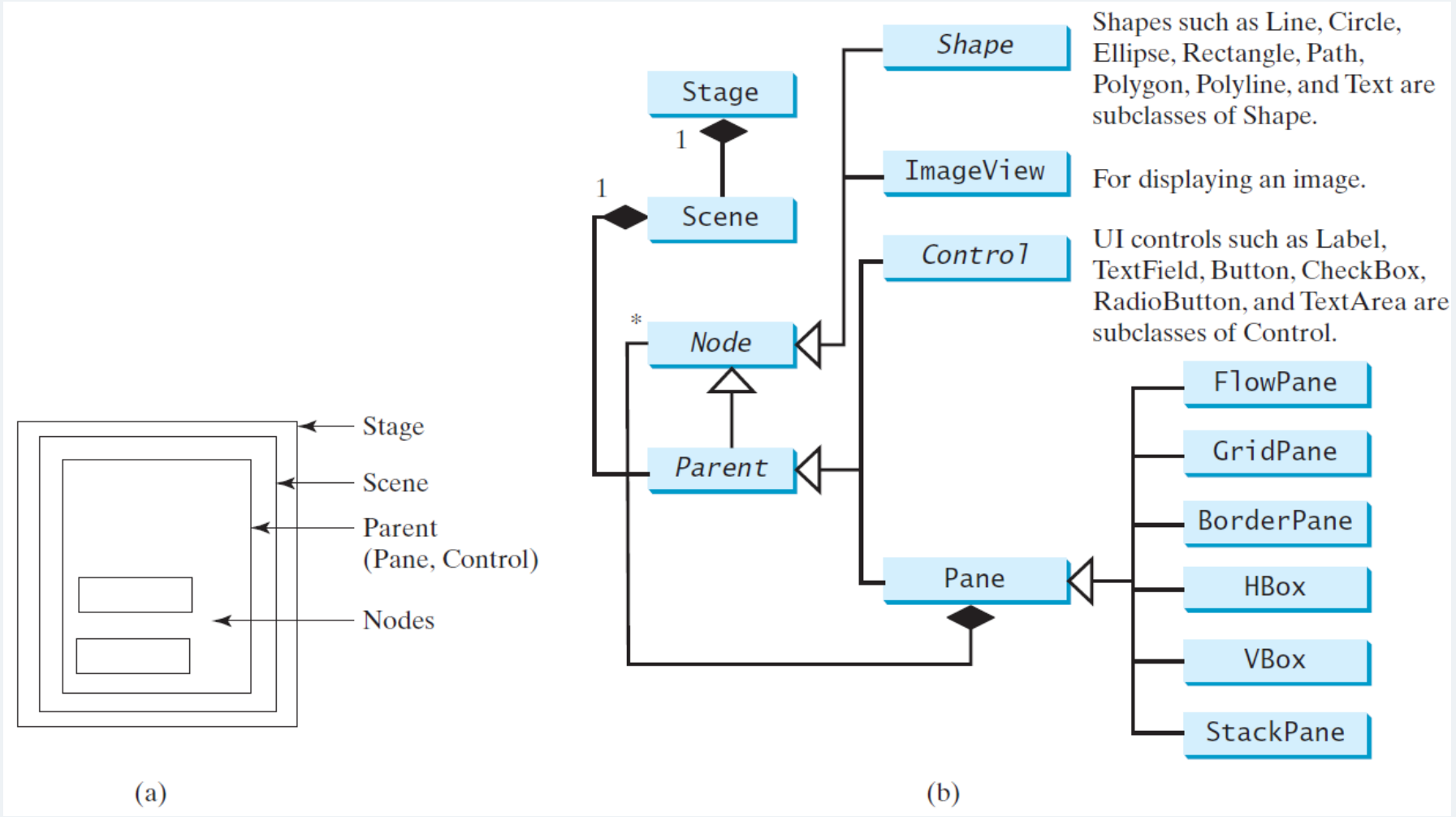
Source:Tutorialspoint

Layout Panes

- Pans Layouts represent how nodes are arranged in a container.
- Choosing the right Layout is 90% responsible for the responsiveness of the UI

Class	*Description*
`Pane`	Base class for layout panes. It contains the `getChildren()` method for returning a list of nodes in the pane.
`StackPane`	Places the nodes on top of each other in the center of the pane.
`FlowPane`	Places the nodes row-by-row horizontally or column-by-column vertically.
`GridPane`	Places the nodes in the cells in a two-dimensional grid.
`BorderPane`	Places the nodes in the top, right, bottom, left, and center regions.
`HBox`	Places the nodes in a single row.
`VBox`	Places the nodes in a single column.

Panes, UI Controls, and Shapes

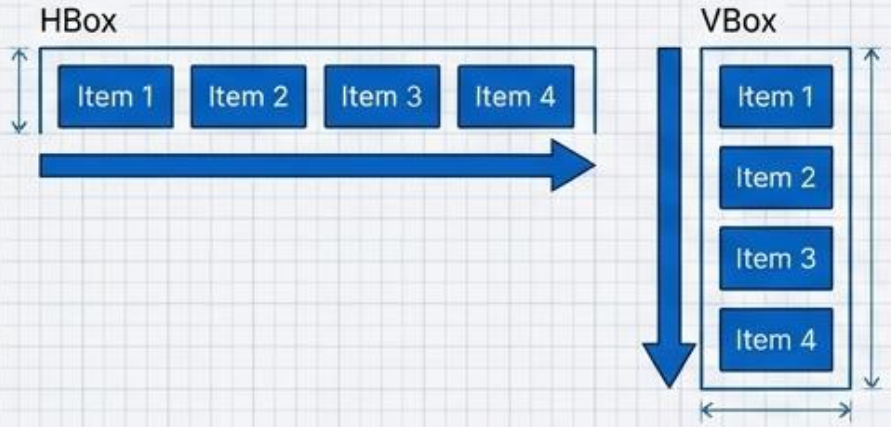


http://docs.oracle.com/javafx/2/scenegraph/jfxpub-scenegraph.htm

Layout Panes

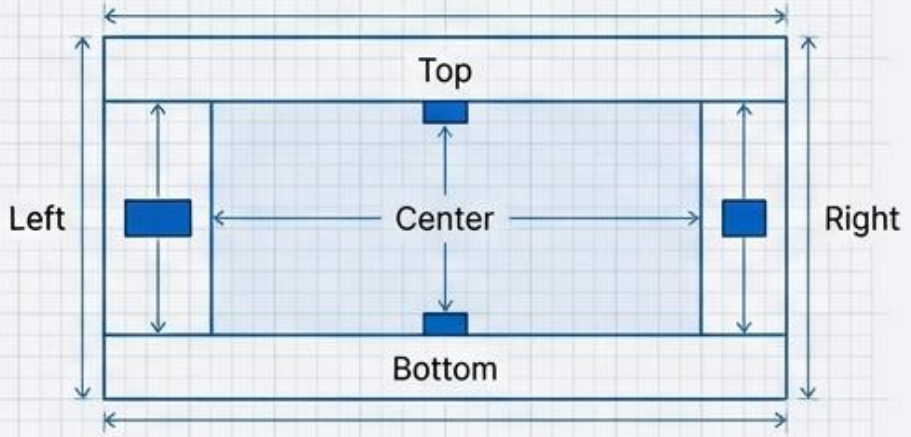
HBox / VBox

JetBrains Mono



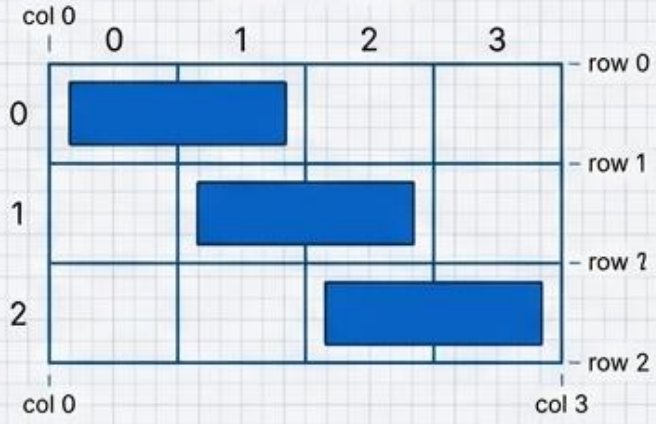
BorderPane

JetBrains Mono



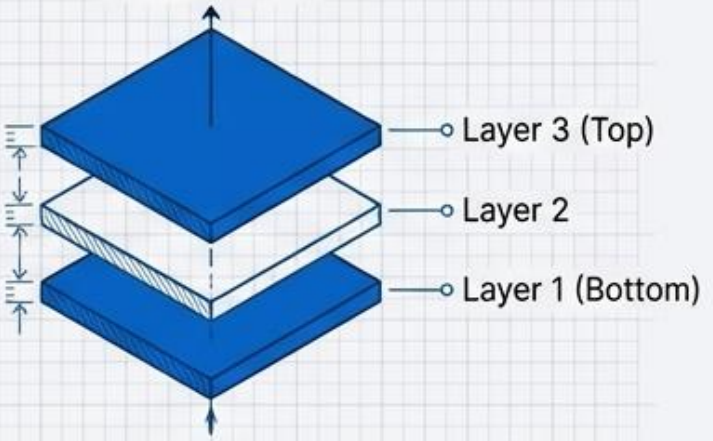
GridPane

JetBrains Mono



StackPane

JetBrains Mono

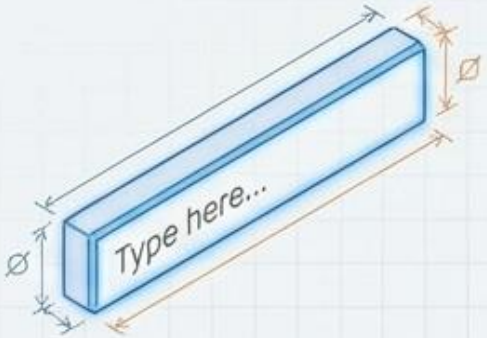


The Toolbox: UI Controls

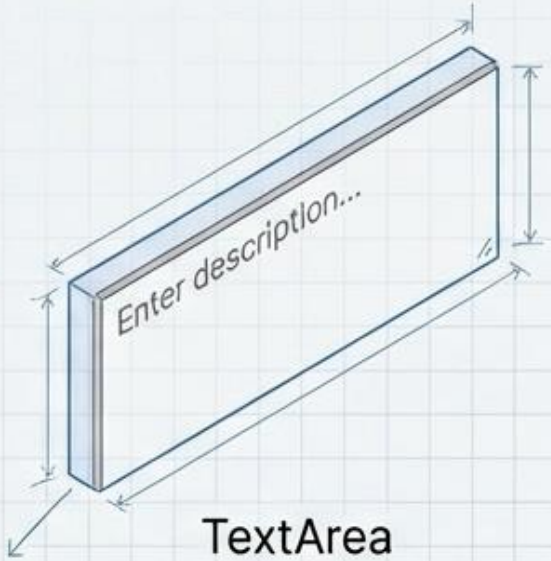
- **Label**
  - **It is a component for displaying text.**
- **Button**
  - **It is a class used for creating buttons.**
- **Menu**
  - **Contains a list of commands or options.**
- **ToolTip**
  - **A pop-up window that displays some additional information about other UI elements.**
- **TextField**
  - **Accepts and displays user input.**

The Toolbox: UI Controls

Component Library

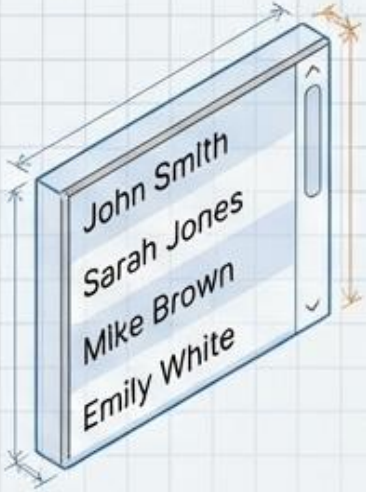


TextField



TextArea

DatePicker

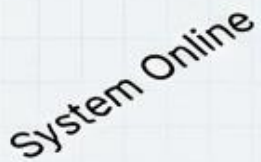


ListView

Button

RadioButton

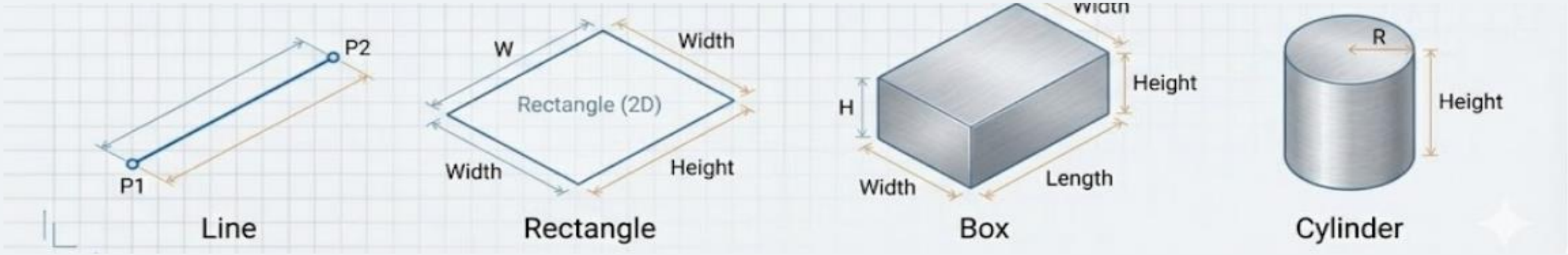
CheckBox



Label

ProgressIndicator

JavaFX Shapes



- **Line**
  - It is a class that represents a line. In general, line is a two-dimensional geometrical shape consist of two points.
- **Rectangle**
  - It is a class used for creating a 2D rectangular shape. In mathematical terms, rectangle is a four-sided polygon.
- **Box**
  - This JavaFX class represents a three-dimensional shape having length, width and height.
- **Cylinder**
  - It is a JavaFX class used to create a Cylinder. In general, cylinder is a closed solid figure that has two properties namely radius and height.

JavaFX Shapes

- **Line**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Line;
import javafx.stage.Stage;

public class LineExample extends Application {

    @Override
    public void start(Stage stage) {
        Line line = new Line(50, 50, 250, 200);
        line.setStroke(Color.BLUE);
        line.setStrokeWidth(3);

        Pane root = new Pane(line);
        Scene scene = new Scene(root, 300, 250);

        stage.setTitle("Line Example");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

JavaFX Shapes

- **Rectangle**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;

public class RectangleExample extends Application {

    @Override
    public void start(Stage stage) {
        Rectangle rectangle = new Rectangle(100, 75, 200, 100);
        rectangle.setFill(Color.LIGHTGREEN);
        rectangle.setStroke(Color.BLACK);

        Pane root = new Pane(rectangle);
        Scene scene = new Scene(root, 400, 250);

        stage.setTitle("Rectangle Example");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

JavaFX Shapes

- **Box (3D shape)**

```
import javafx.application.Application;
import javafx.scene.*;
import javafx.scene.paint.Color;
import javafx.scene.paint.PhongMaterial;
import javafx.scene.shape.Box;
import javafx.stage.Stage;
```

```
public class BoxExample extends Application {

    @Override
    public void start(Stage stage) {
        Box box = new Box(150, 150, 150);

        PhongMaterial material = new PhongMaterial();
        material.setDiffuseColor(Color.CORNFLOWERBLUE);
        box.setMaterial(material);

        Group root = new Group(box);

        Scene scene = new Scene(root, 400, 400, true);
        scene.setCamera(new PerspectiveCamera());

        stage.setTitle("Box Example");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

JavaFX Shapes

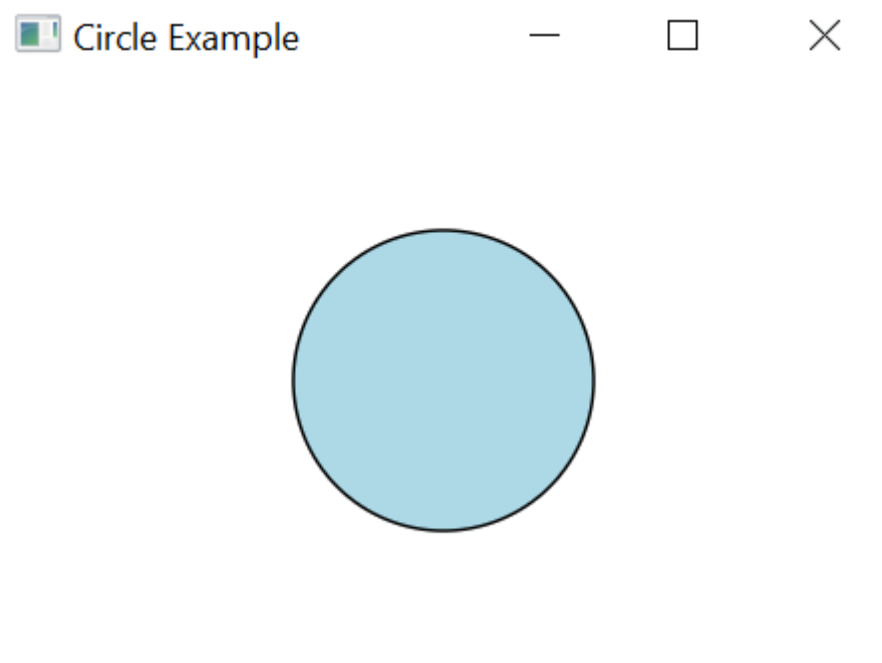
- **Cylinder**

```
import javafx.application.Application;
import javafx.scene.*;
import javafx.scene.paint.Color;
import javafx.scene.paint.PhongMaterial;
import javafx.scene.shape.Cylinder;
import javafx.stage.Stage;
```

```
public class CylinderExample extends Application {

    @Override
    public void start(Stage stage) {
        Cylinder cylinder = new Cylinder(80, 200);

        PhongMaterial material = new PhongMaterial();
        material.setDiffuseColor(Color.ORANGE);
        cylinder.setMaterial(material);

        Group root = new Group(cylinder);

        Scene scene = new Scene(root, 400, 400, true);
        scene.setCamera(new PerspectiveCamera());

        stage.setTitle("Cylinder Example");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

JavaFX Shapes

- **Circle**



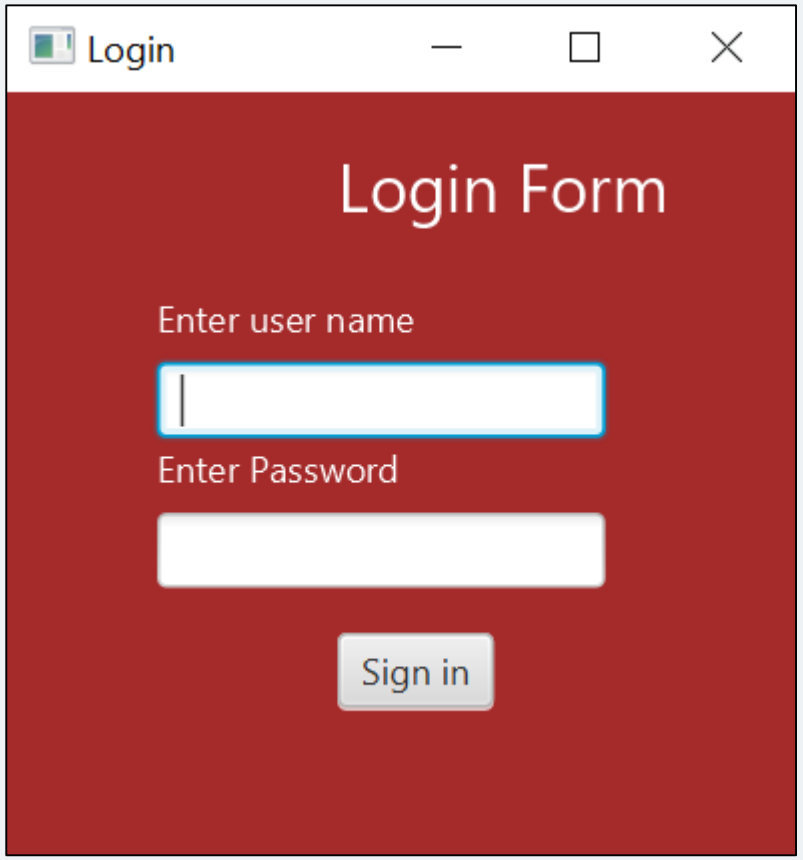

```
import javafx.application.Application;
import javafx.scene.*;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.stage.Stage;

public class CircleExample extends Application {

    @Override
    public void start(Stage stage) {

        Circle circle = new Circle(150, 100, 50);
        circle.setFill(Color.LIGHTBLUE);
        circle.setStroke(Color.BLACK);

        Group root = new Group(circle);
        Scene scene = new Scene(root, 300, 200);

        stage.setTitle("Circle Example");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

JavaFX Text

```
import javafx.application.Application;
import javafx.scene.*;
import javafx.scene.text.*;
import javafx.stage.Stage;

public class DisplayingText extends Application {

    @Override
    public void start(Stage stage) {

        Text text = new Text(50, 150, "Welcome to JavaFX");
        text.setFont(new Font(45));

        Group root = new Group(text);

        stage.setScene(new Scene(root, 600, 300));
        stage.setTitle(“Welcome Page");
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

Login Form

Create A login form such as the following:



Login Form

```
import javafx.application.Application;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.scene.paint.Color;
import javafx.scene.text.*;
import javafx.stage.Stage;
```

```
public class LoginForm extends Application {

    @Override
    public void start(Stage stage) {
        Text title = new Text(110, 40, "Login Form");
        title.setFill(Color.WHITE);
        title.setFont(Font.font(22));

        Text user = new Text(50, 80, "Enter user name");
        user.setFill(Color.WHITE);

        TextField tf = new TextField();
        tf.setLayoutX(50); tf.setLayoutY(90);

        Text pass = new Text(50, 130, "Enter Password");
        pass.setFill(Color.WHITE);

        PasswordField pf = new PasswordField();
        pf.setLayoutX(50); pf.setLayoutY(140);
```

Login Form

```
        Button btn = new Button("Sign in");
        btn.setLayoutX(110); btn.setLayoutY(180);

        Group root = new Group(title, user, tf, pass, pf,
btn);
        Scene scene = new Scene(root, 300, 230,
Color.BROWN);

        stage.setScene(scene);
        stage.setTitle("Login");
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

Assignment

- Write a JavaFX program that creates a **simple graphical dashboard** with the following requirements:
- **Stage Setup**
  - Create a Stage with a title "JavaFX Node & Group Practice".
  - Set the size of the window to **600x400 pixels**.
- **Scene & Group**
  - Create a Group as the root node.
  - Add a Scene to the stage using this root group.
- **Add Nodes to the Group**
  Inside the root Group, add the following nodes:
  - **Circle**
    - Color: Blue
    - Radius: 50
    - Position: (100, 100)
  - **Rectangle**
    - Color: Green
    - Width: 150, Height: 80
    - Position: (250, 50)
  - **Text**
    - Display your name or a message like "Welcome to JavaFX!"
    - Position: (200, 200)
    - Font size: 20
  - **Button**
    - Label: "Click Me"
    - Position: (250, 300)

Thank you